

Mục lục

Thuật ngữ

Bức tranh tổng thể

Phần 1 - Product Pipeline: sản phẩm vào hệ thống

Nhận sự kiện: bốn tình huống, ba đường xử lý

Năm bước xử lý

1. Chuẩn hoá văn bản
2. Dịch đa ngôn ngữ - Gemini 2.5 Flash
3. Gắn tags - Gemini 2.5 Flash (Smart Tags)
4. Dựng văn bản đầu vào cho embedding
5. Tạo vector - AWS Bedrock Embedding, Cohere multilingual v3

Khi một bước AI không trả về kết quả

Từ đồng nghĩa không tham gia bước nào ở đây

Phần 2 - Adaptive Search: khách tìm kiếm

Đánh giá độ chắc chắn

Gọi AI như thế nào

Trộn hai danh sách kết quả

Câu tìm nào đi đường nào

Lọc theo mức điểm liên quan

Lớp bảo vệ dữ liệu vector

Phần 3 - Vòng đời một thay đổi từ đồng nghĩa

Quan hệ với nhánh tìm bằng AI

Phân biệt với sửa lỗi chính tả

Lưu ý: hai hệ thống từ đồng nghĩa riêng biệt

Báo cáo luồng: từ sản phẩm ở GMS đến kết quả tìm kiếm

Tài liệu mô tả **luồng đi của dữ liệu**: từ lúc sửa sản phẩm ở GMS tới lúc khách hàng gõ tìm kiếm và nhận kết quả.

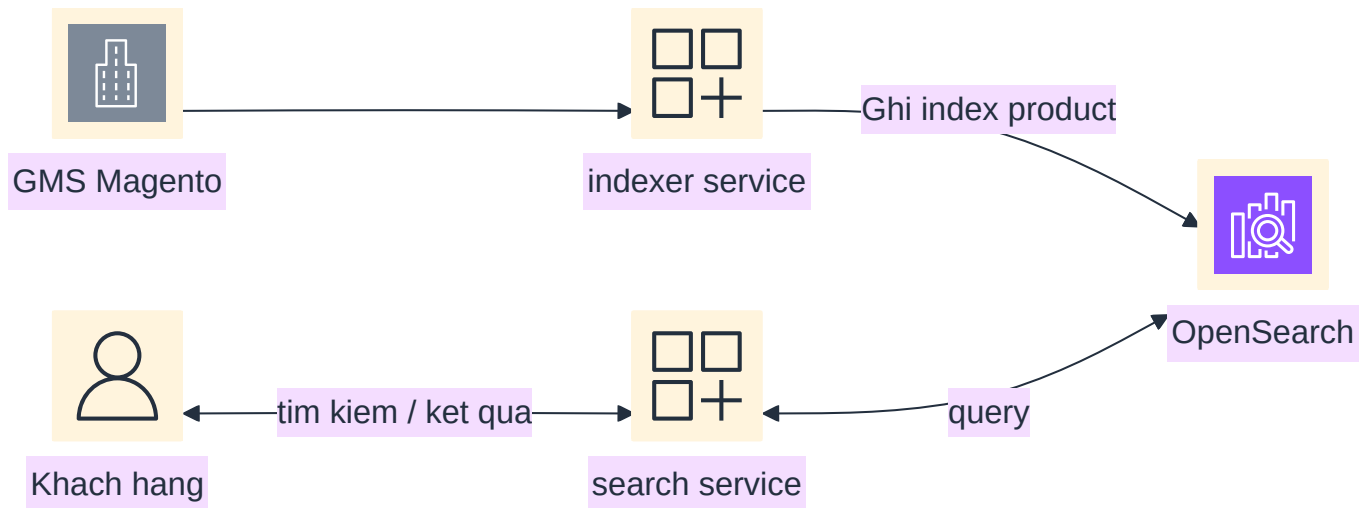
Báo cáo gồm ba phần:

1. **Product Pipeline** - sản phẩm đi vào hệ thống: nhận sự kiện → chuẩn hoá → dịch → gắn tags → dựng văn bản cho embedding → tạo vector → ghi vào kho dữ liệu.
2. **Adaptive Search** - khách gõ tìm kiếm, hệ thống chọn cách trả lời: chỉ dùng từ khoá, hay gọi thêm AI.
3. **Vòng đời từ đồng nghĩa** - cách quản lý bộ từ đồng nghĩa: sửa ở đâu, bấm Áp dụng thì hệ thống làm gì.

Thuật ngữ

- **GMS/Magento**: nơi vận hành sửa sản phẩm.
- **Kafka**: kênh truyền tin nội bộ - GMS báo có thay đổi, hệ thống khác tự lấy về xử lý.
- **indexer-service**: làm giàu dữ liệu sản phẩm (dịch, gắn tags, tạo vector) rồi ghi vào kho tìm kiếm.
- **search-service**: phục vụ tìm kiếm cho khách, đọc từ kho đã chuẩn bị sẵn.
- **OpenSearch**: kho lưu trữ và tìm kiếm của toàn hệ thống.
- **Tìm theo từ khoá (BM25)**: so khớp chữ trong câu tìm với chữ trong sản phẩm.
- **Tìm theo AI (semantic search)**: so khớp ý nghĩa, tìm được cả khi khách gõ khác chữ.
- **Embedding**: bước "dịch" văn bản thành dãy số để AI so sánh ý nghĩa.
- **Enrichment**: dùng AI bổ sung thông tin sản phẩm còn thiếu trước khi ghi vào kho.
- **Trộn kết quả**: hợp nhất kết quả từ khoá và AI thành một danh sách.
- **Độ chắc chắn**: điểm hệ thống tự tính, đo mức chắc chắn của kết quả từ khoá.
- **Từ đồng nghĩa (synonym)**: bảng khai báo các từ cùng nghĩa (VD `thịt lợn` = `thịt heo`) để khách gõ cách nào cũng ra kết quả.
- **Alias**: tên cố định trỏ vào index đang phục vụ; đổi alias sang index khác là cách thay dữ liệu mà khách không bị gián đoạn.

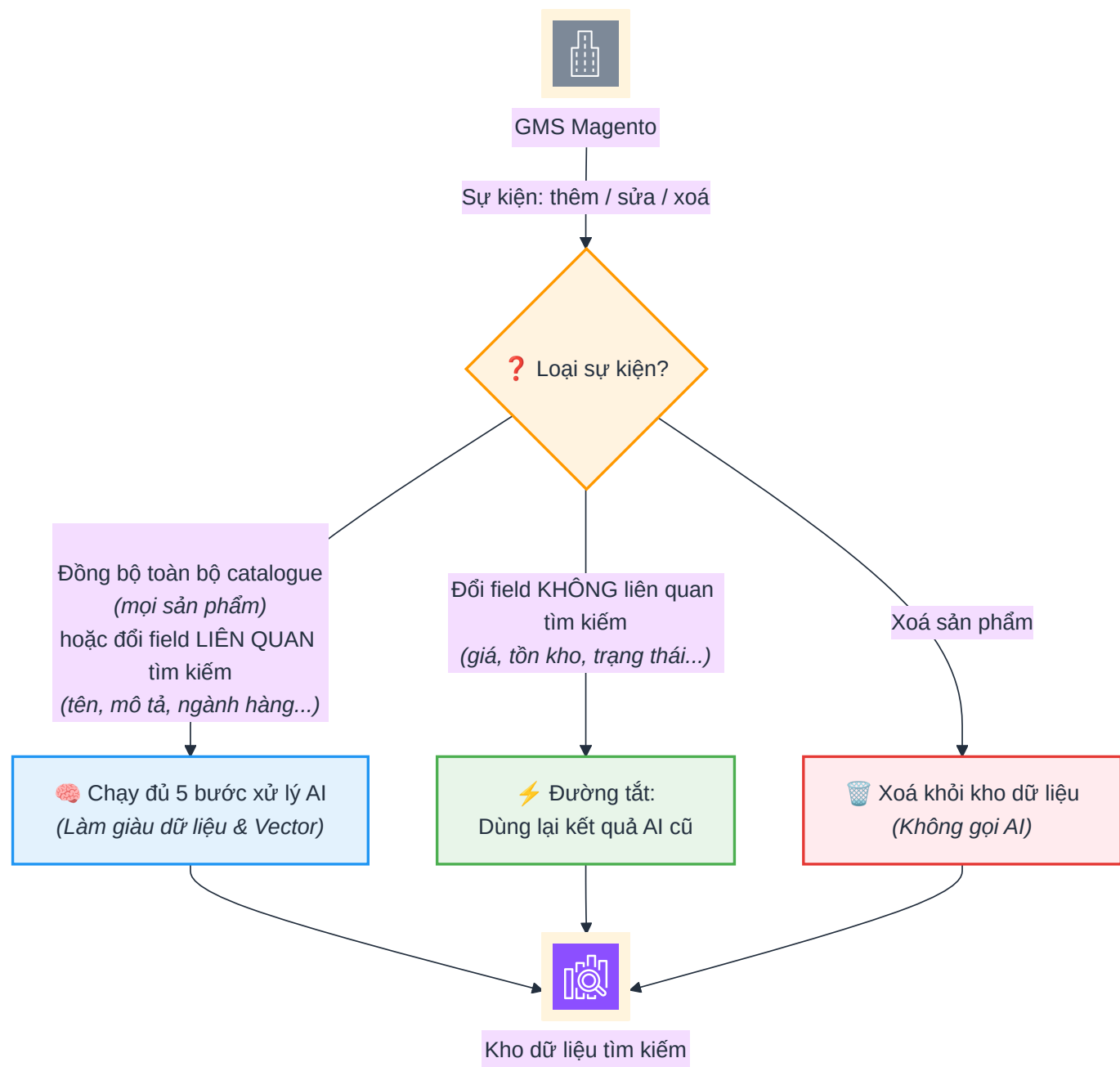
Bức tranh tổng thể



Hình 1 - Bức tranh tổng thể. GMS gửi thay đổi sản phẩm sang indexer-service để ghi vào OpenSearch; khách hàng tìm kiếm qua search-service, đọc từ chính OpenSearch đó.

Phần 1 - Product Pipeline: sản phẩm vào hệ thống

Tóm tắt: một sản phẩm đổi ở GMS đi qua 5 bước xử lý trước khi vào kho dữ liệu tìm kiếm. Ba khái niệm hay được nhắc riêng - "Product Indexing", "Enrich Data", "Smart Tags" - thực ra là các bước của cùng một luồng này.



Hình 2 - Định tuyến một sự kiện thêm/sửa sản phẩm. Bốn tình huống, ba đường xử lý - nhánh trên nhận hai tình huống khác nhau ("đồng bộ toàn bộ catalogue" và "đổi field liên quan tìm kiếm") vì cả hai đều phải chạy đủ 5 bước.

Nhận sự kiện: bốn tình huống, ba đường xử lý

GMS báo thay đổi qua Kafka:

- **Đồng bộ toàn bộ catalogue** - chạy đủ 5 bước cho mọi sản phẩm.
- **Cập nhật field liên quan tìm kiếm** (tên, mô tả, ngành hàng...) - chạy lại đủ 5 bước, vì mọi thứ AI đã sinh ra đều dựa trên các field này.
- **Cập nhật field không liên quan tìm kiếm** (giá, tồn kho, trạng thái...) - đi đường tắt: dùng lại kết quả AI cũ, không gọi AI. Đây là loại thay đổi thường gặp nhất nên đường tắt tiết kiệm đáng kể thời gian và chi phí.
- **Xoá sản phẩm** - xoá khỏi kho dữ liệu, không gọi AI.

GMS luôn gửi toàn bộ thông tin sản phẩm mỗi lần có thay đổi; hệ thống so với lần xử lý gần nhất để biết field nào đổi và chọn đường. Nếu một sự kiện đổi cả hai loại field, hệ thống ghi tạm một bản rồi chạy đủ các bước AI trước khi xác nhận đã xử lý xong.

Năm bước xử lý



Hình 3 - Năm bước chạy bên trong indexer-service. Nhận sự kiện từ GMS, chạy tuần tự 5 bước rồi ghi kết quả vào OpenSearch. Bước 1 thuần xử lý văn bản, bước 2–4 gọi Gemini 2.5 Flash, bước 5 gọi AWS Bedrock (Cohere multilingual v3).

Dữ liệu gửi đi và nhận về ở từng bước:

Bước	Gửi đi	Nhận về
2. Dịch	Tên + mô tả tiếng Việt VD: "Sốt Trộn Salad American Caesar Kuehne 250ml"	Tên + mô tả tiếng Anh, tiếng Hàn
3. Gắn tags	Tên + mô tả đã dịch, kèm danh sách tags cho phép của ngành hàng	Tags phù hợp VD: Sốt Salad, Gia Vị, Hàng Nhập Khẩu
4. Dựng văn bản	Thông tin sản phẩm đã làm giàu ở bước 2-3	Một đoạn văn bản cô đọng, đúng trọng tâm sản phẩm
5. Tạo vector	Đoạn văn bản ở bước 4	Vector 1024 số

1. Chuẩn hoá văn bản

Làm sạch văn bản sản phẩm (bỏ HTML thừa, chuẩn hoá cách viết) để các bước sau không phải xử lý mã định dạng.

2. Dịch đa ngôn ngữ - Gemini 2.5 Flash

Dịch tên và mô tả sản phẩm sang tiếng Anh và tiếng Hàn, để khách gõ bằng những ngôn ngữ đó cũng ra đúng sản phẩm.

3. Gắn tags - Gemini 2.5 Flash (Smart Tags)

AI đọc tên và mô tả **đã dịch** rồi chọn tags từ một danh sách duyệt sẵn. Danh sách này **riêng theo từng ngành hàng**, không phải một danh sách chung cho toàn catalogue - AI chỉ được chọn trong danh sách của ngành hàng mà sản phẩm thuộc về.

AI chỉ nhận tên + mô tả đã dịch + danh sách tags cho phép; không nhận giá, thương hiệu hay thuộc tính thô nào khác.

Sau đó hệ thống lọc lại, chỉ giữ tag nằm trong danh sách cho phép của ngành hàng. Nếu không xác định được ngành hàng, tag do AI đề xuất được lưu thẳng.

4. Dựng văn bản đầu vào cho embedding

Hệ thống ghép các thông tin đã có thành **một đoạn văn bản duy nhất** - thứ AI "đọc" để hiểu sản phẩm. Thành phần điển hình: tên sản phẩm, ngành hàng, tags, thương hiệu, đơn vị đóng gói.

AI chỉ hiểu sản phẩm qua đoạn văn bản này nên chọn đưa gì vào rất quan trọng: thừa chữ ít đặc trưng thì loãng trọng tâm, thiếu thì mất thông tin phân biệt các biến thể. Thành phần đưa vào là cấu hình được.

5. Tạo vector - AWS Bedrock Embedding, Cohere multilingual v3

Biến đoạn văn bản ở bước 4 thành một dãy số (vector 1024 chiều) để máy so sánh được **ý nghĩa**, không chỉ so chữ. Không có bước này, hệ thống chỉ tìm được bằng từ khoá.

Khi một bước AI không trả về kết quả

Nếu một bước (ví dụ dịch) lỗi, sản phẩm vẫn được lưu với phần đã xử lý xong: vẫn tìm được bằng từ khoá, phần còn thiếu được bổ sung ở lần xử lý sau. Trường hợp duy nhất không lưu là tên sản phẩm rỗng sau khi chuẩn hoá.

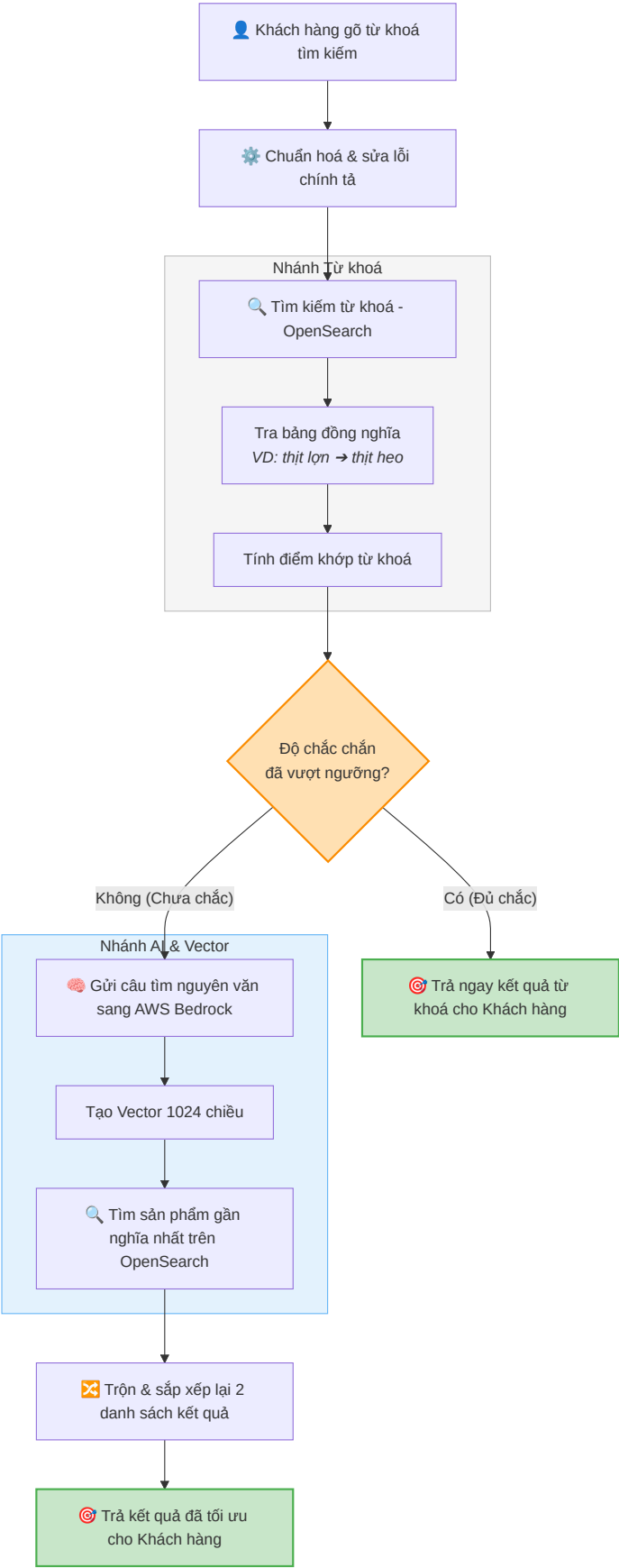
Từ đồng nghĩa không tham gia bước nào ở đây

Từ đồng nghĩa (`thịt lợn` = `thịt heo`) **không nằm trong 5 bước trên**. Nó chỉ áp dụng lúc khách gõ câu tìm: sản phẩm trong kho giữ nguyên tên gốc, không bị "nhân bản" thêm các cách gọi khác.

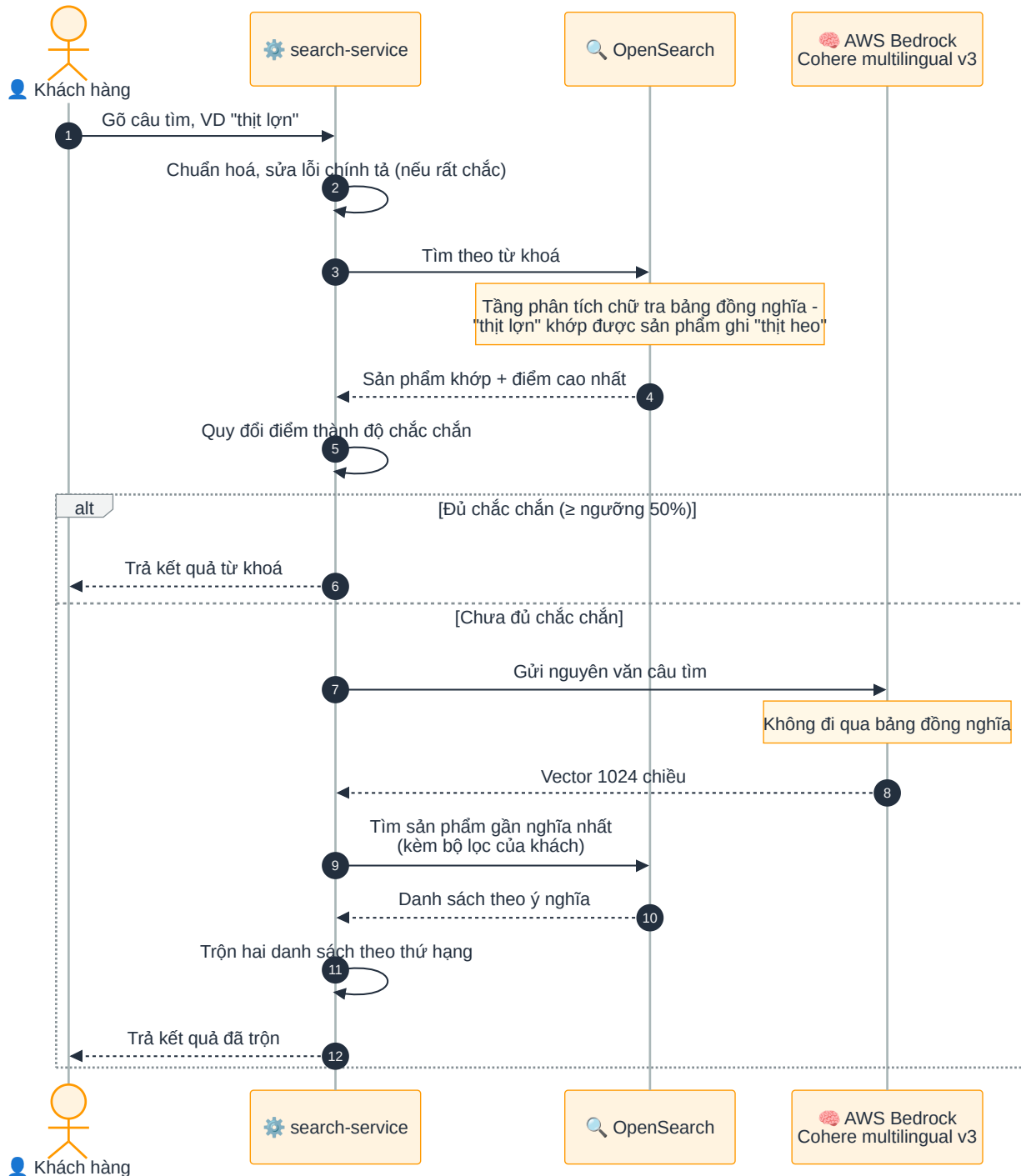
Hệ quả: đổi bộ từ đồng nghĩa không cần chạy lại 5 bước và không tốn thêm lần gọi AI nào. Chi tiết vòng đời quản lý từ đồng nghĩa nằm ở [Phần 3](#).

Phần 2 - Adaptive Search: khách tìm kiếm

Tóm tắt: tìm theo từ khoá luôn chạy trước; hệ thống tự đánh giá độ chắc chắn của kết quả; đủ chắc chắn thì dừng lại, chưa đủ thì gọi thêm AI và trộn hai danh sách - càng ít chắc chắn ở từ khoá thì AI càng được nghe nhiều.



Hình 4 - Một lượt tìm kiếm của khách, kèm chỗ từ đồng nghĩa được áp dụng. Từ khoá luôn chạy trước; chỉ khi độ chắc chắn dưới ngưỡng, hệ thống mới gọi thêm AI rồi trộn hai danh sách. Nhánh từ khoá đi qua bảng đồng nghĩa (nên "thịt lợn" khớp được sản phẩm ghi "thịt heo"), nhánh AI nhận nguyên văn câu khách gõ.



Hình 4.1 - Cùng luồng đó, nhìn theo trình tự gọi giữa các dịch vụ. Hình 4 cho thấy *quyết định* đi đường nào; hình này cho thấy *ai gọi ai*: nhánh chỉ từ khoá dừng sau một vòng sang OpenSearch, còn nhánh có AI phải thêm một vòng sang Bedrock và một vòng k-NN nữa - đây là lý do nhánh AI chậm hơn.

Đánh giá độ chắc chắn

Tìm theo từ khoá chấm điểm mỗi sản phẩm theo **mức độ đặc trưng** của từ khớp được - từ càng hiếm thì khớp được từ đó càng đáng tin. Hệ thống lấy điểm của sản phẩm khớp tốt nhất rồi quy sang thang 0-100%: khớp càng rõ thì % càng gần 100; khớp lẫn lộn nhiều sản phẩm không liên quan thì càng gần 0.

"Đủ chắc chắn" là gì: % này được so với một **ngưỡng** - mức tối thiểu để tin dùng kết quả từ khoá mà không cần hỏi AI. **Ngưỡng hiện đặt ở 0,5 (50%):** từ 50% trở lên thì dừng ở từ khoá, dưới 50% thì gọi thêm AI.

Ngưỡng là cấu hình, chỉnh được mà không cần sửa code - đặt cao hơn thì gọi AI nhiều hơn; đặt thấp hơn thì nhanh hơn và ít tốn chi phí AI hơn.

Gọi AI như thế nào

Khi độ chắc chắn dưới ngưỡng, hệ thống làm hai việc:

1. **Chuyển câu tìm thành vector** - dùng **đúng model** đã tạo vector cho sản phẩm ở Phần 1 (Cohere multilingual v3, 1024 chiều); khác duy nhất là một tham số báo cho model biết đang xử lý *câu tìm* hay *nội dung sản phẩm*. Bắt buộc dùng chung model: hai bên phải cùng một không gian vector thì so sánh ý nghĩa mới có nghĩa.
2. **Tìm những sản phẩm có vector gần nghĩa nhất** - so khoảng cách giữa các dãy số, nên tìm được cả khi câu tìm dùng từ khác hẳn catalogue.

Các bộ lọc của khách (cửa hàng, ngành hàng, thương hiệu, khoảng giá, còn hàng) được áp dụng ngay trong bước tìm theo AI, không lọc sau - lọc sau sẽ gạt mất phần lớn kết quả tốt.

Tìm theo AI luôn trả về "sản phẩm gần nhất về ý nghĩa", không có khái niệm "không tìm thấy gì phù hợp". Nên hệ thống thêm một lớp bảo vệ: kết quả vẫn phải chứa ít nhất một từ khoá chính trong câu tìm.

Trộn hai danh sách kết quả

Hai cách tìm chấm điểm theo hai thang khác nhau nên không cộng trực tiếp được. Hệ thống dùng **trộn theo thứ hạng** (Reciprocal Rank Fusion): xét **vị trí** của mỗi sản phẩm trong từng danh sách thay vì điểm số thô. Mức ưu tiên nghiêng theo độ chắc chắn - từ khoá càng kém chắc chắn, AI càng có tiếng nói lớn hơn.

Câu tìm nào đi đường nào

Quy luật chung: câu tìm càng **chỉ đích danh một mặt hàng** thì càng dừng ở từ khoá; câu tìm càng **mô tả nhu cầu chung chung** hoặc dùng từ lạ với catalogue thì càng cần AI diễn giải.

Câu tìm	Kiểu câu	Đường đi
coca cola	Tên thương hiệu cụ thể	Chỉ từ khoá
thịt heo	Tên mặt hàng phổ thông	Chỉ từ khoá
bia	Một từ ngắn, khớp rõ nhóm hàng	Chỉ từ khoá
đồ ăn cho bé	Mô tả nhu cầu, không phải tên hàng	Từ khoá + AI
nước giặt thơm lâu	Mô tả đặc tính mong muốn	Từ khoá + AI
sữa tươi nhập khẩu	Toàn từ phổ biến trong một ngành hàng	Từ khoá + AI
sua nhập khẩu	Gõ không dấu	Từ khoá + AI
rau mơ	Từ hiếm, catalogue gần như không có	Từ khoá + AI

Có công cụ nội bộ để xem lại một câu tìm: đã đi đường nào, độ chắc chắn bao nhiêu, mỗi sản phẩm do nhánh nào tìm ra.

Lọc theo mức điểm liên quan

Ngoài ngưỡng quyết định "có gọi AI hay không", hệ thống còn có thể loại kết quả AI có điểm liên quan quá thấp trước khi trộn. Đây là nút điều chỉnh riêng, tách biệt với ngưỡng nói trên.

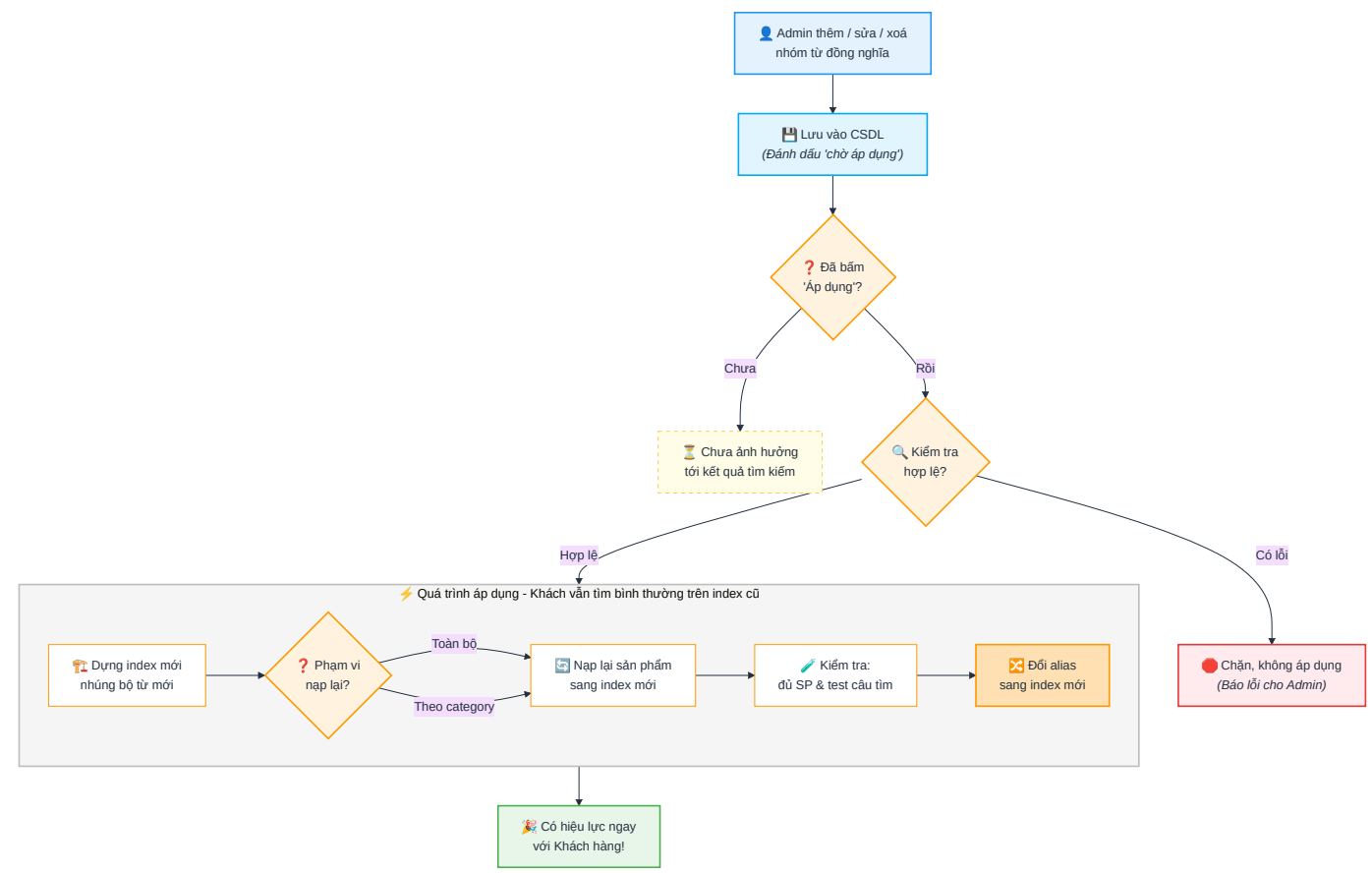
Lớp bảo vệ dữ liệu vector

Hệ thống tự kiểm tra vector trong kho có khớp model AI đang cấu hình không. Nếu lệch, nó âm thầm quay về chỉ dùng từ khoá thay vì trả kết quả có thể sai; khách không bị gián đoạn.

Phần 3 - Vòng đời một thay đổi từ đồng nghĩa

Từ đồng nghĩa được áp dụng **lúc khách gõ** (Hình 4). Phần này trả lời câu hỏi vận hành: sửa ở đâu, và bấm "Áp dụng" thì hệ thống làm gì.

Phạm vi: áp dụng cho **tiếng Việt**, trên các trường tên, mô tả, mô tả ngắn, ngành hàng và tags.



Hình 5 - Vòng đời một thay đổi từ đồng nghĩa. Sửa xong chưa có hiệu lực ngay; phải bấm Áp dụng để hệ thống dựng index mới rồi đổi alias.

Điểm hay bị hiểu nhầm: **có nạp lại dữ liệu, nhưng khách không bị gián đoạn**. Bộ từ được nhúng thẳng vào cấu hình index nên phải dựng index mới; trong lúc dựng, khách vẫn tìm bình thường trên index cũ. Bước cuối đổi alias sang index mới trong một thao tác duy nhất. Cần quay lui thì trở alias ngược lại, vì index cũ vẫn còn.

Bước nạp lại có hai phạm vi: **toàn bộ catalogue**, hoặc **chỉ những category bị ảnh hưởng** khi nhóm từ vừa sửa chỉ liên quan tới một vài ngành hàng - nạp theo category nhanh hơn đáng kể trên catalogue lớn.

Dù chọn phạm vi nào, đây chỉ là chép lại document đã có sang index mới - không chạy lại 5 bước ở Phần 1, không tốn thêm lần gọi AI nào.

Trước khi áp dụng, hệ thống kiểm tra vài luật để chặn lỗi thường gặp: nhóm quá ít từ; từ trùng với từ dừng tiếng Việt (khai vào cũng không bao giờ khớp); nhóm bắc cầu quá rộng, dễ trộn lẫn mặt hàng không liên quan; và nhóm trùng lặp.

Quan hệ với nhánh tìm bằng AI

Bộ từ đồng nghĩa nằm trong tầng xử lý chữ của OpenSearch. Nhánh từ khoá đi qua tầng đó nên được hưởng lợi; nhánh AI nhận nguyên câu khách gõ nên **không đi qua bảng đồng nghĩa**.

Có một công tắc cấu hình để thay từ vùng miền sang từ chuẩn *trước khi* đưa câu tìm cho AI. Hai nguyên tắc: từ chuẩn do người vận hành chọn (không suy theo tần suất, vì một từ có thể phổ biến với nghĩa khác), và chỉ thay đúng một từ chứ không đưa cả nhóm đồng nghĩa vào câu tìm.

Phân biệt với sửa lỗi chính tả

Hai cơ chế khác nhau, hay bị gộp làm một:

- **Từ đồng nghĩa** - khách gõ đúng, chỉ là dùng từ khác.
- **Sửa lỗi chính tả** - khách gõ sai. Hệ thống chấm độ chắc chắn của từ thay thế: rất chắc thì tự sửa và tìm theo câu đã sửa; tương đối chắc thì chỉ gợi ý ("có phải bạn muốn tìm...") và vẫn tìm theo câu khách gõ.

Lưu ý: hai hệ thống từ đồng nghĩa riêng biệt

`search-service` có bộ từ đồng nghĩa riêng (mô tả ở trên); `indexer-service` cũng có một bộ riêng với API và bảng dữ liệu riêng. Hai dịch vụ, hai cơ sở dữ liệu - sửa bên này không ảnh hưởng bên kia.